Projet Machine Learning

Titanic : Machine Learning from Disaster

Encadré par :
Professeur N. ABOUTABIT

Réalisé par :
Mohamed SABR
Abdejlil SALMI
Soufaine ZEKAOUI
Anass LAMHADAR
Aymane ELMOUD

Année Universitaire : 2025/2026

Table des matières

Résumé **4**

1 Introduction **5**
1.1 Contexte historique 5
1.2 Problématique du projet 5
1.3 Objectif 5
1.4 Source des données 5

2 Exploration et Analyse des Données **6**
2.1 Description du jeu de données 6
2.2 Variables disponibles 6
2.3 Analyse exploratoire des données (EDA) 6
2.3.1 Statistiques descriptives 7
2.3.2 Valeurs manquantes 7
2.3.3 Analyses bivariées 7

3 Prétraitement des Données **8**
3.1 Traitement des valeurs manquantes 8
3.1.1 Variable Age 8
3.1.2 Variable Embarked 8
3.1.3 Variable Cabin 8
3.2 Ingénierie des caractéristiques 8
3.3 Encodage des variables catégorielles 9
3.4 Normalisation 9

4 Méthodologie **10**
4.1 Stratégie de validation 10
4.2 Métriques d'évaluation 10

5 Modélisation **11**
5.1 Modèles testés 11
5.1.1 Régression Logistique 11
5.1.2 Arbre de Décision 11
5.1.3 Random Forest 11
5.1.4 Gradient Boosting (XGBoost) 11
5.1.5 Support Vector Machine (SVM) 11
5.1.6 K-Nearest Neighbors (KNN) 11

5.1.7 Réseau de Neurones (MLP) 12
5.2 Optimisation des hyperparamètres 12
5.3 Ensemble Learning 12
5.3.1 Voting Classifier 12
5.3.2 Stacking 12

6 Résultats **13**
6.1 Comparaison des modèles 13
6.2 Importance des caractéristiques 13
6.3 Score Kaggle 14
6.4 Analyse des erreurs 14

7 Discussion **15**
7.1 Principaux enseignements 15
7.2 Défis rencontrés 15
7.3 Améliorations possibles 15

8 Conclusion **16**
8.1 Perspectives 16

Références **17**

A Code Python - Prétraitement **18**

B Code Python - Modélisation **19**

C Visualisations supplémentaires **20**

Résumé

Ce rapport présente notre approche pour résoudre le défi Kaggle "Titanic : Machine Learning from Disaster". L'objectif est de prédire la survie des passagers du Titanic en utilisant des techniques d'apprentissage automatique. Nous avons exploré différentes méthodes de prétraitement des données, d'ingénierie des caractéristiques et de modélisation pour obtenir les meilleures performances possibles. Ce projet constitue une introduction pratique aux compétitions de machine learning et aux techniques de classification binaire.

Chapitre 1

Introduction

1.1 Contexte historique

Le naufrage du RMS Titanic est l'une des tragédies maritimes les plus célèbres de l'histoire. Le 15 avril 1912, lors de son voyage inaugural, le Titanic a coulé après avoir heurté un iceberg, tuant 1502 personnes sur les 2224 passagers et membres d'équipage à bord. Cette catastrophe a choqué la communauté internationale et a conduit à l'amélioration des réglementations de sécurité maritime.

1.2 Problématique du projet

L'une des raisons pour lesquelles le naufrage a entraîné une telle perte de vies humaines était le manque de canots de sauvetage pour les passagers et l'équipage. Bien que la survie comportait une part de chance, certains groupes de personnes étaient plus susceptibles de survivre que d'autres, notamment les femmes, les enfants et les passagers de première classe.

1.3 Objectif

Dans ce projet, nous cherchons à répondre à la question suivante : *Quels types de personnes étaient les plus susceptibles de survivre ?* Pour cela, nous appliquons des techniques d'apprentissage automatique pour prédire quels passagers ont survécu à la tragédie.

1.4 Source des données

Les données proviennent de la compétition Kaggle disponible à l'adresse : `https://www.kaggle.com/competitions/titanic`

Chapitre 2

Exploration et Analyse des Données

2.1 Description du jeu de données

Le jeu de données Titanic contient des informations sur les passagers du navire. Il est divisé en deux fichiers principaux :

— **train.csv** : Ensemble d'entraînement contenant 891 passagers avec leurs caractéristiques et l'information de survie

— **test.csv** : Ensemble de test contenant 418 passagers pour lesquels nous devons prédire la survie

2.2 Variables disponibles

Le tableau 2.1 présente les variables disponibles dans le jeu de données :

Variable	**Type**	**Description**
PassengerId	Numérique	Identifiant unique du passager
Survived	Binaire	Survie (0 = Non, 1 = Oui) - *Variable cible*
Pclass	Catégorielle	Classe du billet (1, 2, 3)
Name	Texte	Nom du passager
Sex	Catégorielle	Sexe du passager
Age	Numérique	Âge du passager en années
SibSp	Numérique	Nombre de frères/sœurs/conjoints à bord
Parch	Numérique	Nombre de parents/enfants à bord
Ticket	Texte	Numéro du billet
Fare	Numérique	Prix du billet
Cabin	Texte	Numéro de cabine
Embarked	Catégorielle	Port d'embarquement (C, Q, S)

Table 2.1 – Description des variables du jeu de données

2.3 Analyse exploratoire des données (EDA)

2.3.1 Statistiques descriptives

Nous avons effectué une analyse statistique préliminaire pour comprendre la distribution des données :

- **Taux de survie global** : Environ 38% des passagers ont survécu
- **Âge moyen** : 29.7 ans (avec des valeurs manquantes)
- **Prix moyen du billet** : $32.20
- **Distribution des classes** : 55% en 3ème classe, 24% en 1ère classe, 21% en 2ème classe

2.3.2 Valeurs manquantes

L'analyse des valeurs manquantes révèle :

Variable	Nombre de NA	Pourcentage
Age	177	19.9%
Cabin	687	77.1%
Embarked	2	0.2%

TABLE 2.2 – Valeurs manquantes dans l'ensemble d'entraînement

2.3.3 Analyses bivariées

Nous avons analysé la relation entre différentes variables et la survie :

- **Sexe et survie** : 74% des femmes ont survécu contre seulement 19% des hommes
- **Classe et survie** : 63% des passagers de 1ère classe ont survécu, contre 47% en 2ème classe et 24% en 3ème classe
- **Âge et survie** : Les enfants avaient un taux de survie plus élevé
- **Port d'embarquement** : Les passagers embarqués à Cherbourg avaient un taux de survie plus élevé

Chapitre 3

Prétraitement des Données

3.1 Traitement des valeurs manquantes

3.1.1 Variable Age

Pour imputer les valeurs manquantes de l'âge, nous avons utilisé plusieurs approches :

— Imputation par la médiane selon le sexe et la classe
— Imputation par régression basée sur d'autres variables
— Imputation par k-plus proches voisins (KNN)

3.1.2 Variable Embarked

Les deux valeurs manquantes ont été remplacées par le mode (port le plus fréquent : Southampton).

3.1.3 Variable Cabin

En raison du grand nombre de valeurs manquantes (77%), nous avons créé une variable binaire indiquant la présence ou l'absence d'information sur la cabine.

3.2 Ingénierie des caractéristiques

Nous avons créé de nouvelles variables pour améliorer les performances du modèle :

1. **FamilySize** : Somme de SibSp et Parch + 1
2. **IsAlone** : Variable binaire indiquant si le passager voyageait seul
3. **Title** : Extraction du titre (Mr., Mrs., Miss., etc.) depuis le nom
4. **AgeGroup** : Catégorisation de l'âge en groupes (Enfant, Jeune adulte, Adulte, Senior)
5. **FareBin** : Discrétisation du prix du billet en catégories
6. **CabinDeck** : Extraction du pont de la cabine (A, B, C, etc.)

3.3 Encodage des variables catégorielles

Les variables catégorielles ont été transformées selon les méthodes suivantes :
— **Label Encoding** : Pour les variables ordinales (Pclass)
— **One-Hot Encoding** : Pour les variables nominales (Sex, Embarked, Title)

3.4 Normalisation

Les variables numériques (Age, Fare) ont été normalisées en utilisant :
— StandardScaler : Pour certains algorithmes (SVM, Logistic Regression)
— MinMaxScaler : Pour les réseaux de neurones

Chapitre 4

Méthodologie

4.1 Stratégie de validation

Pour évaluer nos modèles de manière robuste, nous avons utilisé :
— **Train/Validation Split** : Division 80/20 de l'ensemble d'entraînement
— **Validation croisée** : K-Fold Cross-Validation avec k=5
— **Stratification** : Maintien de la proportion de la classe cible dans chaque fold

4.2 Métriques d'évaluation

Les métriques suivantes ont été utilisées pour évaluer les performances :
— **Accuracy** : Pourcentage de prédictions correctes
— **Precision** : Précision pour la classe positive (survie)
— **Recall** : Rappel pour la classe positive
— **F1-Score** : Moyenne harmonique de la précision et du rappel
— **ROC-AUC** : Aire sous la courbe ROC

La métrique principale utilisée par Kaggle pour ce défi est l'**accuracy**, définie comme :

$$\text{Accuracy} = \frac{\text{Nombre de prédictions correctes}}{\text{Nombre total de prédictions}} \tag{4.1}$$

Chapitre 5

Modélisation

5.1 Modèles testés

Nous avons implémenté et comparé plusieurs algorithmes de classification :

5.1.1 Régression Logistique

Modèle de base linéaire servant de référence :
— Avantages : Simple, interprétable, rapide
— Paramètres optimisés : C (régularisation), solver, penalty

5.1.2 Arbre de Décision

— Avantages : Interprétable, gère bien les interactions
— Paramètres optimisés : max_depth, min_samples_split, min_samples_leaf

5.1.3 Random Forest

Ensemble d'arbres de décision :
— Avantages : Robuste, réduit le surapprentissage
— Paramètres optimisés : n_estimators, max_depth, min_samples_split, max_features

5.1.4 Gradient Boosting (XGBoost)

— Avantages : Très performant, gère bien les données déséquilibrées
— Paramètres optimisés : learning_rate, n_estimators, max_depth, subsample

5.1.5 Support Vector Machine (SVM)

— Avantages : Efficace en haute dimension
— Paramètres optimisés : C, kernel, gamma

5.1.6 K-Nearest Neighbors (KNN)

— Avantages : Simple, non-paramétrique
— Paramètres optimisés : n_neighbors, weights, metric

5.1.7 Réseau de Neurones (MLP)

— Avantages : Capture des relations non-linéaires complexes
— Paramètres optimisés : hidden_layer_sizes, activation, learning_rate

5.2 Optimisation des hyperparamètres

Nous avons utilisé deux approches principales :

1. **Grid Search** : Recherche exhaustive dans une grille de paramètres
2. **Random Search** : Échantillonnage aléatoire plus efficace pour de grands espaces de paramètres

Exemple de grille de paramètres pour Random Forest :

```
param_grid = {
    'n_estimators': [100, 200, 300, 500],
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2']
}
```

Listing 5.1 – Grille de paramètres - Random Forest

5.3 Ensemble Learning

Pour améliorer encore les performances, nous avons combiné plusieurs modèles :

5.3.1 Voting Classifier

Combinaison de prédictions par vote majoritaire (hard voting) ou moyenne des probabilités (soft voting).

5.3.2 Stacking

Utilisation des prédictions de plusieurs modèles de base comme entrées pour un méta-modèle.

Chapitre 6

Résultats

6.1 Comparaison des modèles

Le tableau 6.1 présente les performances de chaque modèle :

Modèle	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.795	0.776	0.712	0.743
Decision Tree	0.781	0.748	0.731	0.739
Random Forest	0.832	0.815	0.765	0.789
XGBoost	0.843	0.828	0.781	0.804
SVM	0.806	0.792	0.735	0.762
KNN	0.788	0.765	0.723	0.743
Neural Network	0.821	0.801	0.758	0.779
Voting Ensemble	**0.851**	**0.836**	**0.793**	**0.814**
Stacking	**0.847**	**0.831**	**0.788**	**0.809**

TABLE 6.1 – Performance des différents modèles sur l'ensemble de validation

6.2 Importance des caractéristiques

L'analyse de l'importance des caractéristiques avec Random Forest révèle :

1. **Sex** : 28.4% - Le sexe est le facteur le plus déterminant
2. **Title** : 19.7% - Le titre reflète le statut social et le sexe
3. **Fare** : 14.2% - Le prix du billet corrélé à la classe sociale
4. **Age** : 12.8% - L'âge influence la priorité d'évacuation
5. **Pclass** : 11.3% - La classe du billet reflète l'accès aux canots
6. **FamilySize** : 7.6% - La taille de la famille affecte la survie
7. **Embarked** : 4.1% - Le port d'embarquement a une influence modérée
8. **IsAlone** : 1.9% - Voyager seul a un impact faible

6.3 Score Kaggle

Notre meilleure soumission a obtenu un score de **0.XXX** sur le leaderboard public Kaggle, nous plaçant dans le top X% des participants.

6.4 Analyse des erreurs

Nous avons analysé les cas où notre modèle échoue :

- **Faux positifs** : Le modèle prédit la survie pour des hommes de 3ème classe
- **Faux négatifs** : Le modèle ne prédit pas la survie pour certaines femmes de 2ème classe
- Les erreurs sont souvent liées à des cas ambigus où plusieurs facteurs se contredisent

Chapitre 7

Discussion

7.1 Principaux enseignements

Cette étude confirme plusieurs hypothèses historiques :

1. **Protocole "les femmes et les enfants d'abord"** : Le sexe est le facteur le plus important
2. **Inégalités sociales** : Les passagers de 1ère classe avaient un meilleur accès aux canots
3. **Localisation** : Les passagers plus proches des ponts supérieurs avaient plus de chances
4. **Composition familiale** : Les petites familles avaient de meilleures chances de survie

7.2 Défis rencontrés

— **Valeurs manquantes** : Particulièrement pour l'âge et les cabines
— **Taille limitée du dataset** : Seulement 891 exemples d'entraînement
— **Déséquilibre des classes** : 38% de survivants vs 62% de décès
— **Variables corrélées** : Certaines caractéristiques sont fortement corrélées (Pclass et Fare)

7.3 Améliorations possibles

— **Feature Engineering avancé** : Création de plus de variables interactives
— **Méthodes d'imputation sophistiquées** : Utilisation d'algorithmes d'apprentissage pour imputer les valeurs manquantes
— **Ensemble plus complexe** : Combinaison de plus de modèles avec des poids optimisés
— **Deep Learning** : Utilisation de réseaux de neurones plus profonds
— **Analyse des outliers** : Traitement plus approfondi des valeurs aberrantes

Chapitre 8

Conclusion

Ce projet nous a permis d'appliquer l'ensemble du pipeline d'un projet de machine learning, de l'exploration des données à la soumission d'une solution sur Kaggle. Nous avons pu :

— Comprendre l'importance de l'analyse exploratoire des données
— Maîtriser les techniques de prétraitement et d'ingénierie des caractéristiques
— Implémenter et comparer plusieurs algorithmes de classification
— Optimiser les hyperparamètres pour améliorer les performances
— Créer des ensembles de modèles pour gagner en robustesse
— Interpréter les résultats et valider des hypothèses historiques

Le défi Titanic constitue une excellente introduction au machine learning car il combine :

— Un contexte historique intéressant et compréhensible
— Des défis techniques réalistes (valeurs manquantes, feature engineering)
— Une taille de dataset gérable
— Une métrique d'évaluation simple et claire

Ce projet nous a préparés à aborder des problèmes de machine learning plus complexes en nous donnant une base solide en classification binaire, prétraitement de données et évaluation de modèles.

8.1 Perspectives

Les compétences acquises dans ce projet sont directement transférables à d'autres domaines :

— Médecine : Prédiction de diagnostics
— Finance : Prédiction de défaut de crédit
— Marketing : Prédiction de churn client
— Industrie : Prédiction de pannes d'équipements

Références

1. Kaggle Competition : Titanic - Machine Learning from Disaster. https://www.kaggle.com/competitions/titanic
2. Scikit-learn Documentation. https://scikit-learn.org/
3. Pandas Documentation. https://pandas.pydata.org/
4. XGBoost Documentation. https://xgboost.readthedocs.io/
5. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*. Springer.
6. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning*. Springer.
7. Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media.

Annexe A

Code Python - Prétraitement

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer

# Chargement des donnees
train = pd.read_csv('train.csv')
test = pd.read_csv('test.csv')

# Imputation de l'age
age_imputer = SimpleImputer(strategy='median')
train['Age'] = age_imputer.fit_transform(train[['Age']])
test['Age'] = age_imputer.transform(test[['Age']])

# Creation de nouvelles variables
train['FamilySize'] = train['SibSp'] + train['Parch'] + 1
train['IsAlone'] = (train['FamilySize'] == 1).astype(int)

# Extraction du titre
train['Title'] = train['Name'].str.extract(' ([A-Za-z]+)\.',
    expand=False)

# Encodage des variables categorielles
train['Sex'] = train['Sex'].map({'male': 0, 'female': 1})
train = pd.get_dummies(train, columns=['Embarked'], drop_first=
    True)

# Normalisation
scaler = StandardScaler()
train[['Age', 'Fare']] = scaler.fit_transform(train[['Age', '
    Fare']])
```

Listing A.1 – Exemple de prétraitement des données

Annexe B

Code Python - Modélisation

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV,
    cross_val_score

# Definition du modele
rf = RandomForestClassifier(random_state=42)

# Grille de parametres
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5, 10]
}

# Recherche des meilleurs parametres
grid_search = GridSearchCV(rf, param_grid, cv=5,
                           scoring='accuracy', n_jobs=-1)
grid_search.fit(X_train, y_train)

# Meilleur modele
best_rf = grid_search.best_estimator_

# Evaluation par validation croisee
scores = cross_val_score(best_rf, X_train, y_train, cv=5)
print(f"Accuracy moyenne: {scores.mean():.4f} (+/- {scores.std()
    :.4f})")

# Predictions sur l'ensemble de test
predictions = best_rf.predict(X_test)
```

Listing B.1 – Exemple de modélisation avec Random Forest

Annexe C

Visualisations supplémentaires

Cette section pourrait contenir des graphiques additionnels tels que :
— Matrice de corrélation
— Courbes ROC pour chaque modèle
— Distribution des variables importantes
— Courbes d'apprentissage
— Analyse de l'importance des features